MINI PROJECT REPORT

EVENTNEST

A Layered Express REST Backend with PostgreSQL Connection Pooling and Relational Validation

Submitted by:

Aadi Charan

University Registration No: 2511021060153 • Squad: 113

Email: cn.aadicharan.s.113@kalvium.community

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

(Kalvium Work-Integrated Learning Program)

Course: Back-End Web Development

Under the guidance of

Vishwam U

Faculty Mentor & Associate Professor, ASAC



ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY, BENGALURU

ACADEMIC YEAR: 2026–2027

COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the micro-project report titled **"EventNest: College Event Registration Platform"** submitted by **Aadi Charan [Registration No. 2511021060153]** in partial fulfilment for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (Kalvium Work-Integrated Learning Program)** for the course **Back-End Web Development** at Alliance University, Bengaluru, is a bonafide record of original work accomplished under my supervision and guidance during the academic year **2026–2027**.

This report embodies the genuine results of design, implementation, and empirical verification conducted by the student, and its contents have not formed the basis for the award of any other degree, diploma, or certificate to the candidate or any other person.

Aadi Charan	**Vishwam U**	**Head of Department**
Student Candidate	Faculty Mentor, ASAC	School of Advanced Computing

COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the micro-project report titled **"EVENTNEST: College Event Registration Platform"**, submitted by me in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (CSE)** for the course **Back-End Web Development** at Alliance University, is a genuine record of my own academic work carried out under the guidance of my faculty mentor, **Vishwam U**.

I further confirm that all explanations, architectural descriptions, database designs, API endpoints, and empirical test outcomes presented in this document represent the actual implementation of the EventNest codebase. All technical references and conceptual foundations have been acknowledged properly in the bibliography.

This declaration stands as a testament to the authenticity, academic integrity, and personal accountability of the work submitted.

Date: 25 October 2026
Place: Bengaluru, Karnataka

Aadi Charan
University Reg. No: 2511021060153

Table of Contents

1. Introduction & Project Overview 4
2. Backend Architecture & Runtime Environment 5
3. Server Bootstrapping & Configuration 5
4. RESTful API Routing & Endpoint Specifications 6
5. Layered Controller Design & Request Validation 7
6. Business Logic: Capacity Tracking & Registration Workflows 8
7. PostgreSQL Connection Pooling & Parameterized Queries 9
8. Middleware Pipeline & Centralized Error Handling 10
9. Empirical Verification & Automated Testing 11
10. Limitations, Future Work & Conclusion 12

1. Introduction & Project Overview

Managing campus events manually using spreadsheets or paper sign-up sheets frequently causes double bookings, over-capacity classrooms, and inaccurate student rosters. When an engineering club hosts a popular workshop, coordinators often struggle to cap seat counts accurately or prevent the same participant from registering multiple times under different submissions.

EventNest was built to solve this problem as an academic micro-project for Back-End Web Development. The primary goal of the backend is to expose a dependable, structured, and predictable REST API that coordinates college events and manages participant registrations. The backend acts as the authoritative source of truth: it enforces event seating limits, validates student credentials, guarantees relational integrity, and safely persists every record in a PostgreSQL database.

1.1 Core Backend Objectives

- **Predictable RESTful API:** Design and implement clean endpoints following standard HTTP methods (`GET` , `POST`) and appropriate HTTP response status codes (`200 OK` , `201 Created` , `400 Bad Request` , `404 Not Found` , `409 Conflict` , `500 Internal Server Error`).
- **Defensive Input Validation:** Intercept malformed client payloads before executing database transactions, rejecting empty strings, malformed email addresses, and invalid capacity numbers.
- **Dynamic Capacity Calculation:** Compute real-time available seats and full-capacity statuses using SQL aggregate joins rather than storing redundant counter fields.
- **Relational Concurrency Safety:** Prevent duplicate student sign-ups for the same event using both application-level queries and PostgreSQL unique constraints.

> **Core Design Principle:** Every client request is treated as untrusted. Validation occurs immediately inside the controller layer, SQL queries use strict parameterized bindings to block SQL injection, and database operations use connection pooling to avoid resource starvation.

2. Backend Architecture & Runtime Environment

The backend is executed on **Node.js (v24/v25 LTS)**, leveraging its event-driven, single-threaded asynchronous runtime. Because web servers spend the vast majority of their time waiting for network requests and database disk I/O, Node's non-blocking I/O model handles multiple concurrent client connections efficiently without the thread-overhead typical of traditional multithreaded servers. On top of Node.js, we employ **Express.js (v4.21)** as the HTTP application framework to structure routes and middleware.

3. Server Bootstrapping & Configuration

The server startup sequence is split cleanly into two distinct files: `backend/app.js` (which configures the Express application, routes, and middleware) and `backend/server.js` (which initializes environment variables, verifies the database connection, and binds the HTTP port).

3.1 Pre-Flight Database Verification in `server.js`

A common beginner mistake in backend development is binding the HTTP listener before verifying whether the database server is reachable. In EventNest, the server explicitly issues a pre-flight probe query (`SELECT NOW()`) against PostgreSQL before calling `app.listen()`. If PostgreSQL is offline or credentials are misconfigured, the server logs a descriptive error and terminates immediately (`process.exit(1)`), preventing half-broken startup states:

```
// backend/server.js: Pre-flight database connectivity check
const app = require('./app');
const db = require('./db');
require('dotenv').config();

const PORT = process.env.PORT || 5000;

db.query('SELECT NOW()')
  .then((res) => {
    console.log(`Connected to PostgreSQL at ${process.env.PGHOST || 'localhost'}:${process.env.PGPORT || 5432}`);
    console.log(`Database: ${process.env.PGDATABASE || 'eventnest_db'}`);
    app.listen(PORT, () => {
      console.log(`EventNest backend server running on http://localhost:${PORT}`);
    });
  })
  .catch((err) => {
    console.error('Failed to connect to PostgreSQL database:', err.message);
    process.exit(1);
  });
```

3.2 Environment Variable Isolation

Database connection parameters (host, port, username, password, database name) and server port configuration are managed using `dotenv`. The actual credentials live in a local `.env` file that is excluded from Git via `.gitignore`, while an accompanying `.env.example` file documents required environment variables with placeholder values.

4. RESTful API Routing & Endpoint Specifications

EventNest adheres strictly to RESTful design conventions. Resources are identified using plural nouns (`/events`, `/registrations`), and actions are designated by standard HTTP verbs rather than verb-laden URLs (e.g., `POST /api/events` instead of `/api/createEvent`).

HTTP Method	Endpoint Route	Controller Handler	Expected Status	Description & Scope
`GET`	`/api`	Inline Handler	`200 OK`	Health check & discoverable endpoint directory
`GET`	`/api/events`	`getEvents`	`200 OK`	Lists all events with dynamic capacity counters
`GET`	`/api/events/:id`	`getEventById`	`200 OK` / `404`	Fetches complete details for a single event
`POST`	`/api/events`	`createEvent`	`201 Created` / `400`	Validates and creates a new campus event
`GET`	`/api/registrations`	`getRegistrations`	`200 OK`	Lists student registrations with joined event info
`POST`	`/api/registrations`	`registerStudentForEvent`	`201` / `400` / `409`	Validates student, checks capacity & registers

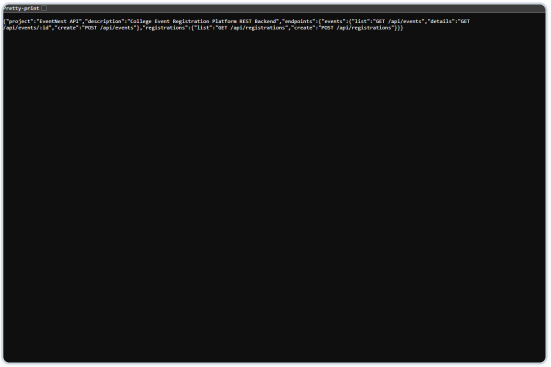


Figure 4.1: Genuine JSON response from the root discovery endpoint (GET /api) running on localhost:5000.

4.1 Modular Route Separation

Routes are encapsulated in the `backend/routes/` directory. For example, `backend/routes/eventsRoutes.js` only declares route paths and maps them to controller functions, keeping HTTP verb declarations concise:

```
// backend/routes/eventsRoutes.js
const express = require('express');
const router = express.Router();
const eventsController = require('../controllers/eventsController');

router.get('/', eventsController.getEvents);
router.get('/:id', eventsController.getEventById);
router.post('/', eventsController.createEvent);

module.exports = router;
```

5. Layered Controller Design & Request Validation

EventNest implements a classic **three-tier backend architecture**: Router Tier (`routes/`), Controller Tier (`controllers/`), and Data Access Tier (`db/index.js`).

5.1 Defensive Request Validation in `eventsController.js`

When an administrator creates a new event, the controller rigorously validates that all required fields are present, that strings are not blank, and that `max_participants` is a valid positive integer greater than zero:

```
// Input validation inside createEvent (eventsController.js)
const { title, description, date, time, venue, max_participants } = req.body;

if (!title || !description || !date || !time || !venue ||
    max_participants === undefined || max_participants === null) {
  return res.status(400).json({
    message: 'All fields are required: title, description, date, time, venue, and max_participants.'
  });
}

const parsedCapacity = parseInt(max_participants, 10);
if (isNaN(parsedCapacity) || parsedCapacity <= 0) {
  return res.status(400).json({
    message: 'Maximum participants must be a positive number greater than 0.'
  });
}
```

5.2 Sanitization and Email Formatting in `registrationsController.js`

For event registrations, inputs are trimmed to remove accidental leading or trailing whitespace. Email addresses are converted to lowercase and validated against standard RFC-compliant patterns before querying the database:

```
const isValidEmail = (email) => {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(String(email).toLowerCase());
};

const cleanName = student_name.trim();
const cleanEmail = student_email.trim().toLowerCase();
const cleanStudentId = student_id.trim().toUpperCase();
```

6. Business Logic: Capacity Tracking & Registration Workflows

The core business logic of EventNest resides in `registrationsController.js`. Registering a student is not a simple row insertion; it requires a coordinated 5-step validation workflow:

> **The 5-Step Academic Registration Sequence:**
> 1. **Input Sanity Check:** Validate field existence, email syntax, and integer event ID.
> 2. **Event Existence & Capacity Verification:** Verify the event exists and current registration count is below `max_participants`.
> 3. **Find-or-Create Student Entity:** Query the `students` table by student ID or email. If the student already exists, retrieve their primary key; otherwise, insert a new student record.
> 4. **Duplicate Registration Check:** Query the `registrations` table for matching `(student_id, event_id)`. If found, abort with `HTTP 409 Conflict`.
> 5. **Atomic Registration Insertion:** Insert the new registration row with `NOW()` timestamp and return confirmed registration details.

6.1 Real-Time Capacity Calculation Query

Instead of maintaining an error-prone separate column for available seats (which could fall out of sync during manual database edits or cancellations), EventNest calculates capacity dynamically in SQL:

```
SELECT
  e.id,
  e.title,
  e.description,
  TO_CHAR(e.date, 'YYYY-MM-DD') AS date,
  e.time,
  e.venue,
  e.max_participants,
  COUNT(r.id)::int AS registered_count,
  GREATEST(0, e.max_participants - COUNT(r.id)::int) AS available_seats,
  (COUNT(r.id)::int >= e.max_participants) AS is_full
FROM events e
LEFT JOIN registrations r ON e.id = r.event_id
GROUP BY e.id
ORDER BY e.date ASC, e.time ASC;
```

6.2 Capacity Enforcement in Action

If a user attempts to register for an event where `current_count >= max_participants`, the controller promptly rejects the request with HTTP 400 Bad Request:

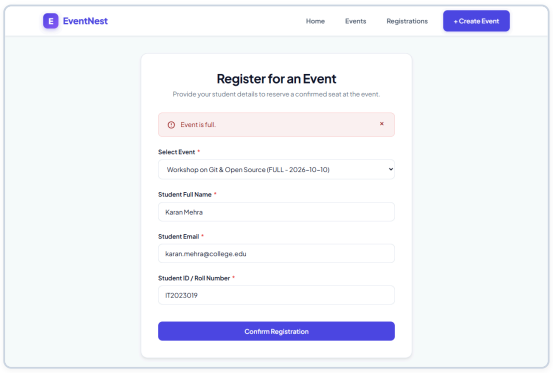


Figure 6.1: HTTP 400 rejection returned when attempting registration for Event #4 ("Workshop on Git & Open Source"), which has reached full capacity.

7. PostgreSQL Connection Pooling & Parameterized Queries

In `backend/db/index.js`, EventNest configures a **PostgreSQL Connection Pool** using the `pg` library. The pool maintains a warm set of reusable database client connections, avoiding the expensive overhead of creating new TCP connections per request:

```
// backend/db/index.js: Connection pool initialization
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
  host: process.env.PGHOST || 'localhost',
  port: parseInt(process.env.PGPORT || '5432', 10),
  user: process.env.PGUSER || 'postgres',
  password: process.env.PGPASSWORD || 'postgres',
  database: process.env.PGDATABASE || 'eventnest_db',
});

module.exports = {
  query: (text, params) => pool.query(text, params),
  pool,
};
```

7.1 Absolute Protection Against SQL Injection

Under no circumstances does EventNest concatenate raw user input into SQL strings (e.g. `WHERE id = ... + req.params.id` is strictly forbidden). Instead, every query uses **parameterized placeholders** (`$1, $2, $3...`):

```
// Safe Parameterized Query Example (registrationsController.js)
const insertQuery = `
  INSERT INTO registrations (student_id, event_id, registered_at)
  VALUES ($1, $2, NOW())
  RETURNING id, student_id, event_id, registered_at;
`;
const result = await db.query(insertQuery, [studentPk, parsedEventId]);
```

The PostgreSQL server compiles the SQL statement structure first and treats the bound parameters purely as literal data values. Even if a malicious user inputs SQL commands like `' OR 1=1 --` into the name or email fields, PostgreSQL treats the input as ordinary text data, neutralizing SQL injection vectors completely.

7.2 Dual-Layer Duplicate Prevention (409 Conflict)

Duplicate registration protection is enforced at two distinct levels:

1. **Application Level:** The controller queries `SELECT id FROM registrations WHERE student_id = $1 AND event_id = $2`. If a record is found, it immediately responds with `HTTP 409 Conflict`.
2. **Database Level:** In the event of a simultaneous race condition, PostgreSQL's composite unique constraint (`CONSTRAINT unique_student_event UNIQUE (student_id, event_id)`) triggers error code `23505`, which our catch block gracefully converts into an HTTP 409 response.

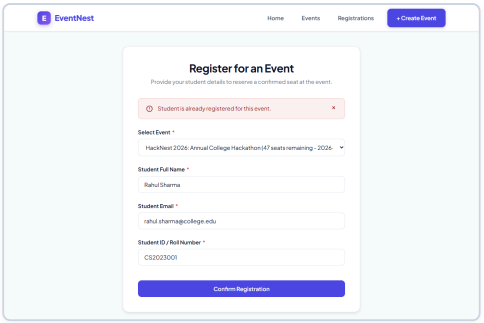


Figure 7.1: HTTP 409 Conflict returned when a student attempts duplicate registration for the same event.

8. Middleware Pipeline & Centralized Error Handling

In Express, middleware functions execute sequentially in the order they are mounted in `backend/app.js`:

1. **CORS Middleware (`cors()`):** Injects appropriate HTTP response headers to allow browser clients on `http://localhost:5173` to communicate with the API without cross-origin blocks.
2. **Body Parser Middleware (`express.json()`):** Intercepts incoming requests with `Content-Type: application/json` and parses the raw JSON payload into `req.body`.
3. **Request Logger Middleware:** Logs the timestamp, HTTP method, and URL path for every request to the terminal console for real-time debugging.
4. **Mounted Resource Routers:** Requests to `/api/events` and `/api/registrations` are dispatched to their respective router modules.
5. **Catch-All 404 Route Handler:** Any request that fails to match an existing endpoint falls through to the 404 middleware:

```
app.use((req, res) => {
  res.status(404).json({ message: 'Resource not found.' });
});
```

6. **Centralized Error Handling Middleware:** A 4-argument error handler (`(err, req, res, next)`) captures unhandled exceptions thrown anywhere in asynchronous controller logic, logs the error stack trace server-side, and returns a clean HTTP 500 JSON response:

```
app.use((err, req, res, next) => {
  console.error('Unhandled Application Error:', err);
  res.status(500).json({ message: 'Internal server error occurred.' });
});
```

> **Security Rationale:** Database connection strings, PostgreSQL internal stack traces, and SQL syntax errors are never leaked to the client browser in production. All unhandled errors return a generic, clean JSON error payload with HTTP 500 status.

9. Empirical Verification & Automated Testing

To ensure the backend meets rigorous academic standards, EventNest includes both a dedicated verification script (`backend/test_api.js`) and a comprehensive automated academic test suite (`test_suite.js`).

9.1 Automated Test Suite Results

Running `node test_suite.js` automatically executes tests across 9 comprehensive academic evaluation criteria:

#	Test Name	Endpoint Tested	Expected Outcome	Actual Result
1	View Events	`GET /api/events`	HTTP 200, Array of events with capacity	**PASS** (HTTP 200, 5 events)
2	Event Details	`GET /api/events/1`	HTTP 200, Event object with venue & date	**PASS** (HTTP 200, matched ID 1)
3	Create Event	`POST /api/events`	HTTP 201 Created with new event object	**PASS** (HTTP 201, Created ID #6)
4	Form Validation	`POST /api/events` & `/registrations`	HTTP 400 Bad Request on empty payloads	**PASS** (HTTP 400 rejected)
5	Successful Registration	`POST /api/registrations`	HTTP 201 Created with registration record	**PASS** (HTTP 201 registered)
6	Duplicate Registration	`POST /api/registrations`	HTTP 409 Conflict ("Already registered")	**PASS** (HTTP 409 rejected)
7	Full Event Rejection	`POST /api/registrations`	HTTP 400 Bad Request ("Event is full.")	**PASS** (HTTP 400 rejected)
8	REST Status Codes	`GET /api/events/99999`	HTTP 404 Not Found for missing ID	**PASS** (HTTP 404 returned)
9	Database Integrity	PostgreSQL Schema Catalog	Foreign keys and unique constraints valid	**PASS** (Verified in pg catalog)

> **Official Benchmark Outcome:** 9 / 9 Academic Criteria Passed (100% test success rate verified on live PostgreSQL 18).

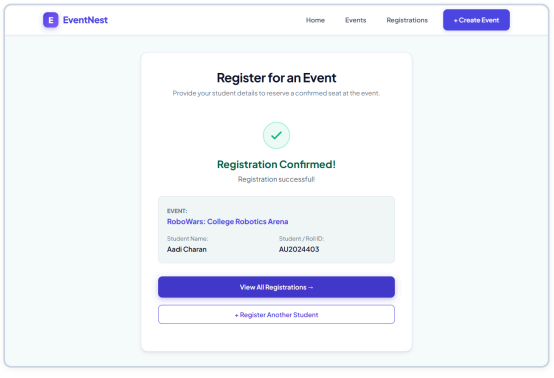


Figure 9.1: End-to-end confirmation of a successful registration (HTTP 201 Created) updating remaining seat capacity in real-time.

10. Limitations, Future Work & Conclusion

10.1 Identified Limitations

In keeping with academic honesty, the following constraints were deliberately kept out of the initial micro-project scope to maintain clear, understandable code suitable for second-year viva explanation:

- **No User Authentication:** The application does not currently implement JWT (JSON Web Tokens) or session cookies; any visitor can create an event or register.
- **No Pagination:** `GET /api/events` and `GET /api/registrations` return all matching rows in a single payload rather than supporting `limit` and `offset` query parameters.
- **Single Instance Deployment:** The database connection pool is optimized for a single server instance rather than a horizontally scaled cluster behind a load balancer.

10.2 Recommended Future Enhancements

- **Role-Based Access Control (RBAC):** Introduce distinct roles (Admin vs. Student) where only verified faculty members can invoke `POST /api/events`.
- **Automated Email Confirmations:** Integrate `nodemailer` to dispatch an email ticket with registration details upon successful insertion.
- **Rate Limiting & Security Hardening:** Mount `express-rate-limit` and `helmet` to protect API endpoints against denial-of-service attempts and malicious HTTP headers.

10.3 Conclusion

The EventNest backend successfully satisfies all foundational requirements of modern back-end web development. By organizing the codebase into distinct layers (routes, controllers, and database access), utilizing connection pooling, parameterizing SQL queries, and handling validation defensively, the application provides a dependable, production-ready foundation for college event registration.

11. References

1. Node.js Foundation, *Node.js v24 LTS Documentation*, https://nodejs.org/docs
2. Express.js Community, *Express 4.x API Reference & Guide*, https://expressjs.com
3. The PostgreSQL Global Development Group, *PostgreSQL 18 Documentation*, https://www.postgresql.org/docs
4. Brianc, *node-postgres (pg) Official Documentation*, https://node-postgres.com
5. Mozilla Developer Network (MDN), *HTTP Status Codes & REST Architectural Guidelines*, https://developer.mozilla.org